
MODULE *ReadReplicaRouting*

Formal specification of the Spring Boot Read-Replica Routing Library.

The library routes *JDBC* queries to either *MASTER* or *REPLICA* based on:

- A per-thread *RoutingContext* (*NONE* | *REPLICA*)
- The health of the replica datasource (*HEALTHY* | *UNHEALTHY*)
- Whether an active write transaction is in progress on the thread

We model *N* threads executing concurrently. Each thread has:

routingCtx : the current *ThreadLocal* routing context
txState : transaction state (*NONE* | *WRITE* | *READ_ONLY*)
priorCtx : the saved context to restore after a *@ReadOnly* scope
inReadOnly : whether the thread is currently inside a *@ReadOnly* scope
lastServedBy : the datasource that served the most recent query

Health state is global (shared across threads, as in the real *DataSourceHealthMonitor* using *AtomicBoolean*).

EXTENDS *Naturals*, *FiniteSets*, *TLC*

Constants

CONSTANT *Threads* finite set of thread ids, *e.g.* { *T1*, *T2* }

Value domains

CtxValues $\triangleq$ { "NONE", "REPLICA" }
TxValues $\triangleq$ { "NONE", "WRITE", "READ_ONLY" }
HealthValues $\triangleq$ { "HEALTHY", "UNHEALTHY" }
DsValues $\triangleq$ { "MASTER", "REPLICA" }

Variables

VARIABLES

routingCtx, *routingCtx*[*t*] : *CtxValues* — *ThreadLocal* for thread *t*
txState, *txState*[*t*] : *TxValues* — active transaction kind
priorCtx, *priorCtx*[*t*] : *CtxValues* — context saved by *EnterReadOnly*
inReadOnly, *inReadOnly*[*t*] : BOOLEAN — inside *@ReadOnly* scope?
lastServedBy, *lastServedBy*[*t*] : *DsValues* — datasource of last query
masterHealth, *masterHealth* : *HealthValues* — global master health flag
replicaHealth *replicaHealth* : *HealthValues* — global replica health flag

vars $\triangleq$ $\langle$ *routingCtx*, *txState*, *priorCtx*, *inReadOnly*, *lastServedBy*,
masterHealth, *replicaHealth* $\rangle$

Type invariant

$$\begin{aligned}
TypeOK &\triangleq \\
&\wedge \text{ routingCtx} \in [Threads \rightarrow CtxValues] \\
&\wedge \text{ txState} \in [Threads \rightarrow TxValues] \\
&\wedge \text{ priorCtx} \in [Threads \rightarrow CtxValues] \\
&\wedge \text{ inReadOnly} \in [Threads \rightarrow \text{BOOLEAN}] \\
&\wedge \text{ lastServedBy} \in [Threads \rightarrow DsValues] \\
&\wedge \text{ masterHealth} \in \text{HealthValues} \\
&\wedge \text{ replicaHealth} \in \text{HealthValues}
\end{aligned}$$

DetermineTarget(*t*)

Pure operator mirroring *RoutingDataSource.determineCurrentLookupKey*():

- *WRITE* tx active $\rightarrow$ always *MASTER* (write transaction wins)
- *routingCtx* = *REPLICA* AND replica *HEALTHY* $\rightarrow$ *REPLICA*
- otherwise $\rightarrow$ *MASTER* (plain call, or replica unhealthy fallback)

Note: *masterHealth* does not appear in this function — master health is observable-only and never influences routing decisions.

$$\begin{aligned}
DetermineTarget(t) &\triangleq \\
&\text{IF } txState[t] = \text{"WRITE"} \\
&\quad \text{THEN "MASTER"} \\
&\quad \text{ELSE IF } routingCtx[t] = \text{"REPLICA"} \wedge replicaHealth = \text{"HEALTHY"} \\
&\quad \quad \text{THEN "REPLICA"} \\
&\quad \quad \text{ELSE "MASTER"}
\end{aligned}$$

$$WriteTransactionActive(t) \triangleq txState[t] = \text{"WRITE"}$$

Initial state

$$\begin{aligned}
Init &\triangleq \\
&\wedge \text{ routingCtx} = [t \in Threads \mapsto \text{"NONE"}] \\
&\wedge \text{ txState} = [t \in Threads \mapsto \text{"NONE"}] \\
&\wedge \text{ priorCtx} = [t \in Threads \mapsto \text{"NONE"}] \\
&\wedge \text{ inReadOnly} = [t \in Threads \mapsto \text{FALSE}] \\
&\wedge \text{ lastServedBy} = [t \in Threads \mapsto \text{"MASTER"}] \\
&\wedge \text{ masterHealth} = \text{"HEALTHY"} \\
&\wedge \text{ replicaHealth} = \text{"HEALTHY"}
\end{aligned}$$

Health monitor actions

Probe detects replica failure

$$\begin{aligned}
\text{ReplicaBecomesUnhealthy} &\triangleq \\
&\wedge \text{replicaHealth} = \text{"HEALTHY"} \\
&\wedge \text{replicaHealth}' = \text{"UNHEALTHY"} \\
&\wedge \text{UNCHANGED } \langle \text{routingCtx}, \text{txState}, \text{priorCtx}, \text{inReadOnly}, \\
&\quad \text{lastServedBy}, \text{masterHealth} \rangle
\end{aligned}$$

Probe detects replica recovery

$$\begin{aligned}
\text{ReplicaRecovers} &\triangleq \\
&\wedge \text{replicaHealth} = \text{"UNHEALTHY"} \\
&\wedge \text{replicaHealth}' = \text{"HEALTHY"} \\
&\wedge \text{UNCHANGED } \langle \text{routingCtx}, \text{txState}, \text{priorCtx}, \text{inReadOnly}, \\
&\quad \text{lastServedBy}, \text{masterHealth} \rangle
\end{aligned}$$

Probe detects master failure (observability only — does not change routing)

$$\begin{aligned}
\text{MasterBecomesUnhealthy} &\triangleq \\
&\wedge \text{masterHealth} = \text{"HEALTHY"} \\
&\wedge \text{masterHealth}' = \text{"UNHEALTHY"} \\
&\wedge \text{UNCHANGED } \langle \text{routingCtx}, \text{txState}, \text{priorCtx}, \text{inReadOnly}, \\
&\quad \text{lastServedBy}, \text{replicaHealth} \rangle
\end{aligned}$$

Probe detects master recovery

$$\begin{aligned}
\text{MasterRecovers} &\triangleq \\
&\wedge \text{masterHealth} = \text{"UNHEALTHY"} \\
&\wedge \text{masterHealth}' = \text{"HEALTHY"} \\
&\wedge \text{UNCHANGED } \langle \text{routingCtx}, \text{txState}, \text{priorCtx}, \text{inReadOnly}, \\
&\quad \text{lastServedBy}, \text{replicaHealth} \rangle
\end{aligned}$$

@ReadOnly aspect actions

Aspect entry — no active write tx: save prior ctx, push *REPLICA*

$$\begin{aligned}
\text{EnterReadOnly}(t) &\triangleq \\
&\wedge \neg \text{inReadOnly}[t] \\
&\wedge \neg \text{WriteTransactionActive}(t) \\
&\wedge \text{priorCtx}' = [\text{priorCtx} \text{ EXCEPT } ![t] = \text{routingCtx}[t]] \\
&\wedge \text{routingCtx}' = [\text{routingCtx} \text{ EXCEPT } ![t] = \text{"REPLICA"}] \\
&\wedge \text{inReadOnly}' = [\text{inReadOnly} \text{ EXCEPT } ![t] = \text{TRUE}] \\
&\wedge \text{UNCHANGED } \langle \text{txState}, \text{lastServedBy}, \text{masterHealth}, \text{replicaHealth} \rangle
\end{aligned}$$

Aspect entry — write tx is active: skip routing change, proceed as-is

$$\text{EnterReadOnlySkipped}(t) \triangleq$$

$$\begin{aligned}
& \wedge \neg inReadOnly[t] \\
& \wedge WriteTransactionActive(t) \\
& \wedge inReadOnly' = [inReadOnly \text{ EXCEPT } ![t] = \text{TRUE}] \\
& \wedge \text{UNCHANGED } \langle routingCtx, txState, priorCtx, lastServedBy, \\
& \quad masterHealth, replicaHealth \rangle
\end{aligned}$$

Aspect finally block — always restores prior context regardless of path taken

$$\begin{aligned}
ExitReadOnly(t) & \triangleq \\
& \wedge inReadOnly[t] \\
& \wedge routingCtx' = [routingCtx \text{ EXCEPT } ![t] = priorCtx[t]] \\
& \wedge inReadOnly' = [inReadOnly \text{ EXCEPT } ![t] = \text{FALSE}] \\
& \wedge priorCtx' = [priorCtx \text{ EXCEPT } ![t] = \text{"NONE"}] \\
& \wedge \text{UNCHANGED } \langle txState, lastServedBy, masterHealth, replicaHealth \rangle
\end{aligned}$$

Transaction lifecycle actions

@Transactional (readOnly = false) — always routes to MASTER

$$\begin{aligned}
BeginWriteTx(t) & \triangleq \\
& \wedge txState[t] = \text{"NONE"} \\
& \wedge txState' = [txState \text{ EXCEPT } ![t] = \text{"WRITE"}] \\
& \wedge \text{UNCHANGED } \langle routingCtx, priorCtx, inReadOnly, lastServedBy, \\
& \quad masterHealth, replicaHealth \rangle
\end{aligned}$$

$$\begin{aligned}
CommitWriteTx(t) & \triangleq \\
& \wedge txState[t] = \text{"WRITE"} \\
& \wedge txState' = [txState \text{ EXCEPT } ![t] = \text{"NONE"}] \\
& \wedge \text{UNCHANGED } \langle routingCtx, priorCtx, inReadOnly, lastServedBy, \\
& \quad masterHealth, replicaHealth \rangle
\end{aligned}$$

@Transactional (readOnly = true) — does not override @ReadOnly routing

$$\begin{aligned}
BeginReadOnlyTx(t) & \triangleq \\
& \wedge txState[t] = \text{"NONE"} \\
& \wedge txState' = [txState \text{ EXCEPT } ![t] = \text{"READ_ONLY"}] \\
& \wedge \text{UNCHANGED } \langle routingCtx, priorCtx, inReadOnly, lastServedBy, \\
& \quad masterHealth, replicaHealth \rangle
\end{aligned}$$

$$\begin{aligned}
CommitReadOnlyTx(t) & \triangleq \\
& \wedge txState[t] = \text{"READ_ONLY"} \\
& \wedge txState' = [txState \text{ EXCEPT } ![t] = \text{"NONE"}] \\
& \wedge \text{UNCHANGED } \langle routingCtx, priorCtx, inReadOnly, lastServedBy, \\
& \quad masterHealth, replicaHealth \rangle
\end{aligned}$$

Query execution

Models a *JDBC* query being issued. *DetermineTarget(t)* is evaluated and its result recorded in *lastServedBy[t]*. This is the only action that observably “chooses” a datasource.

$$\begin{aligned} \textit{ExecuteQuery}(t) &\triangleq \\ &\wedge \textit{lastServedBy}' = [\textit{lastServedBy} \text{ EXCEPT } ![t] = \textit{DetermineTarget}(t)] \\ &\wedge \text{UNCHANGED } \langle \textit{routingCtx}, \textit{txState}, \textit{priorCtx}, \textit{inReadOnly}, \\ &\quad \textit{masterHealth}, \textit{replicaHealth} \rangle \end{aligned}$$

Next-state relation

$$\begin{aligned} \textit{Next} &\triangleq \\ &\vee \textit{ReplicaBecomesUnhealthy} \\ &\vee \textit{ReplicaRecovers} \\ &\vee \textit{MasterBecomesUnhealthy} \\ &\vee \textit{MasterRecovers} \\ &\vee \exists t \in \textit{Threads} : \\ &\quad \vee \textit{EnterReadOnly}(t) \\ &\quad \vee \textit{EnterReadOnlySkipped}(t) \\ &\quad \vee \textit{ExitReadOnly}(t) \\ &\quad \vee \textit{BeginWriteTx}(t) \\ &\quad \vee \textit{CommitWriteTx}(t) \\ &\quad \vee \textit{BeginReadOnlyTx}(t) \\ &\quad \vee \textit{CommitReadOnlyTx}(t) \\ &\quad \vee \textit{ExecuteQuery}(t) \end{aligned}$$

Fairness

WF on *ExecuteQuery*: a thread that can issue a query eventually does.
Required for both liveness properties.

WF on *ReplicaRecovers*: the health monitor eventually detects recovery.
Required for *EventualRecovery* to be non-vacuous.

$$\begin{aligned} \textit{Fairness} &\triangleq \\ &\wedge \forall t \in \textit{Threads} : \text{WF}_{\textit{vars}}(\textit{ExecuteQuery}(t)) \\ &\wedge \text{WF}_{\textit{vars}}(\textit{ReplicaRecovers}) \end{aligned}$$

Specification

$$Spec \triangleq Init \wedge \Box[Next]_{vars} \wedge Fairness$$

Safety invariants

A query is routed to *REPLICA* iff $ctx = REPLICA$ AND replica *HEALTHY* AND no write tx
ReplicaRoutingCorrectness $\triangleq$

$$\begin{aligned} &\forall t \in Threads : \\ &\quad DetermineTarget(t) = \text{"REPLICA"} \\ &\quad \equiv \\ &\quad (routingCtx[t] = \text{"REPLICA"} \\ &\quad \wedge replicaHealth = \text{"HEALTHY"} \\ &\quad \wedge txState[t] \neq \text{"WRITE"}) \end{aligned}$$

An active write transaction always routes to *MASTER*

$$\begin{aligned} &WriteTxAlwaysMaster \triangleq \\ &\quad \forall t \in Threads : \\ &\quad txState[t] = \text{"WRITE"} \Rightarrow DetermineTarget(t) = \text{"MASTER"} \end{aligned}$$

Replica unhealthy $\rightarrow$ all threads fall back to *MASTER*

$$\begin{aligned} &FallbackWhenReplicaUnhealthy \triangleq \\ &\quad replicaHealth = \text{"UNHEALTHY"} \Rightarrow \\ &\quad \forall t \in Threads : DetermineTarget(t) = \text{"MASTER"} \end{aligned}$$

Master health does not appear in *DetermineTarget*; routing is never changed by master health state

$$\begin{aligned} &MasterHealthDoesNotBlockRouting \triangleq \\ &\quad \forall t \in Threads : \\ &\quad DetermineTarget(t) = \\ &\quad \quad \text{IF } txState[t] = \text{"WRITE"} \text{ THEN "MASTER"} \\ &\quad \quad \text{ELSE IF } routingCtx[t] = \text{"REPLICA"} \wedge replicaHealth = \text{"HEALTHY"} \\ &\quad \quad \text{THEN "REPLICA"} \\ &\quad \quad \text{ELSE "MASTER"} \end{aligned}$$

After @ReadOnly scope exits, *priorCtx* is reset to sentinel *NONE*

$$\begin{aligned} &AspectContextCleanup \triangleq \\ &\quad \forall t \in Threads : \\ &\quad \neg inReadOnly[t] \Rightarrow priorCtx[t] = \text{"NONE"} \end{aligned}$$

Liveness properties

While the replica is unhealthy, every thread's ROUTING DECISION (not historical *lastServedBy*) is *MASTER*. This is a safety property expressed as a temporal formula — equivalent to *FallbackWhenReplicaUnhealthy* but

stated here for clarity alongside the recovery property.

Note: *lastServedBy* is a trailing indicator (records the last query's actual target). Using it in a leads-to formula is imprecise because a query may have correctly hit the replica BEFORE the failure; requiring *lastServedBy* to eventually flip to *MASTER* conflates past history with future routing decisions. *DetermineTarget* captures the live decision.

$$\begin{aligned} \text{EventualFallback} &\triangleq \\ &\text{replicaHealth} = \text{"UNHEALTHY"} \Rightarrow \\ &\quad \forall t \in \text{Threads} : \text{DetermineTarget}(t) = \text{"MASTER"} \end{aligned}$$

Once the replica recovers AND a thread has *routingCtx* = *REPLICA* (inside a *@ReadOnly* scope with no write tx), the routing decision for that thread is immediately *REPLICA* — no delay needed, *DetermineTarget* is a pure function evaluated on demand.

$$\begin{aligned} \text{EventualRecovery} &\triangleq \\ &\forall t \in \text{Threads} : \\ &\quad (\text{replicaHealth} = \text{"HEALTHY"} \wedge \\ &\quad \text{routingCtx}[t] = \text{"REPLICA"} \wedge \\ &\quad \text{txState}[t] \neq \text{"WRITE"}) \\ &\quad \Rightarrow \text{DetermineTarget}(t) = \text{"REPLICA"} \end{aligned}$$

Master theorem

$$\begin{aligned} \text{THEOREM } \text{Spec} &\Rightarrow \\ &\square \text{TypeOK} \\ &\wedge \square \text{ReplicaRoutingCorrectness} \\ &\wedge \square \text{WriteTxAlwaysMaster} \\ &\wedge \square \text{FallbackWhenReplicaUnhealthy} \\ &\wedge \square \text{MasterHealthDoesNotBlockRouting} \\ &\wedge \square \text{AspectContextCleanup} \\ &\wedge \text{EventualFallback} \\ &\wedge \text{EventualRecovery} \end{aligned}$$